

Teaching Programming in the Age of AI: Transforming Pedagogy Amidst Code-Generating Technologies

Asad Azemi

Computer Science and Engineering Technology
University of Maryland Eastern Shore
Princess Anne, MD USA
aazemi@umes.edu

Abstract— This innovative to practice full-paper examines how the rapid evolution of artificial intelligence (AI) tools capable of generating code—exemplified by systems such as ChatGPT and GitHub Copilot—has instigated a pedagogical shift in how programming is taught and learned. As these AI tools become more deeply embedded in professional and academic practices, educators are compelled to reassess traditional pedagogical approaches and realign them with the demands of an AI-augmented future. This paper explores the profound implications of these developments for computer programming education, focusing on the challenges that instructors face and the promising opportunities for curricular innovation. While AI tools provide valuable learning assistance, they risk encouraging superficial understanding if students rely on them excessively. We begin by reviewing major challenges associated with the use of AI tools by students, such as the ease with which students can generate complete code solutions without engaging in the problem-solving process. The birth of code-generating AI also poses significant challenges in maintaining academic rigor and assessment fairness. We also cover a short review of strategies and processes that instructors are employing to combat this issue. Finally, we will present our proposed innovative approach, which includes the integration of AI into programming education by shifting the focus from merely producing code to designing and architecting software systems. This approach encourages students to use AI as a collaborative tool, fostering creativity, strategic thinking, and ethical reasoning. This approach requires greater emphasis on systems thinking and design, which can foster an entrepreneurial mindset.

Keywords—*Artificial Intelligence, Large Language Model, ChatGPT, Computer Programming.*

I. INTRODUCTION

Innovation has long been recognized as a primary driver of societal and technological progress. In the context of engineering and computer science education, the integration of artificial intelligence (AI) has opened new avenues for curriculum design, student engagement, and educational research. Among the most significant AI-driven developments are code-generating tools, such as ChatGPT and GitHub Copilot, which leverage Large Language Models (LLMs) to provide on-demand coding assistance [1], [2]. In the rapidly evolving landscape of engineering by instantly generating syntactically valid code, debugging suggestions, and even

explanatory comments, these platforms can drastically alter the way students learn to program. Yet, as these tools become increasingly commonplace in both professional software development and academic settings, educators are challenged to adapt their methodologies and ensure that the depth of student learning remains uncompromised.

The impact of LLM products such as ChatGPT is getting recognized. Researchers and public figures talk about its possible impact. Influential people such as Warren Buffet, renowned investor and chairman of Berkshire Hathaway, compared it to nuclear power, "We let a genie out of the bottle when we developed nuclear weapons," "AI is somewhat similar --- it's part way out of the bottle." His image and voice were recently replicated by an AI-backed tool, and as he said it, they were so convincing that they could have fooled his own family [3]. In this regard, the potential impact of code-generating AI on academic settings is enormous. [4] has examined the complex balance between technological enthusiasm and institutional preparedness, highlighting that educational systems must simultaneously adapt to AI's potential while establishing frameworks to guide its ethical use. Lau and Guo [5] indicate that instructors are split between banning AI tools and fully integrating them into programming pedagogy to align with future job markets. [6] explore how academic institutions are adapting curricula to include AI-based programming tools amid institutional challenges. On one hand, these tools help mitigate common obstacles in learning to program---such as cryptic compilation errors and time-consuming debugging---thereby reducing student frustration and dropout rates [7], [8]. On the other hand, overreliance on AI can lead students to skip key cognitive steps, resulting in superficial understanding of computational concepts. Zawacki-Richter et al. [9] caution that AI-based applications in higher education demand persistent educator involvement to prevent misuse and to foster deep learning. In the realm of programming, this warning translates into a pedagogical necessity: instructors must guide students on how to use AI outputs critically, verifying the accuracy of generated code and reflecting on the reasoning behind each solution.

Moreover, researchers point to growing concerns about academic integrity and assessment fairness when AI can generate entire solutions with minimal human input [10], [11],

[12]. Bali et al. [37] frames these ethical considerations within an "ethics of care" paradigm, emphasizing that institutions should create policies and teaching approaches that protect both the quality of learning and the honesty of student work. Choi et al. [38] illustrates how the same predicament arises in the legal domain, where ChatGPT was found capable of passing law school exams but raised alarms over skill mastery and authenticity in student submissions. Translating these challenges to computer science education, instructors risk losing reliable measures of conceptual understanding if their assignment formats remain static in the face of increasingly sophisticated AI assistance.

Despite these challenges, code-generating AI also offers substantial pedagogical opportunities. Researchers and practitioners alike have noted that AI tools can serve as supplementary "tutors," offering instantaneous feedback on syntax, logical flow, and algorithmic complexity [13], [14]. Such real-time responsiveness can greatly enhance student engagement, a critical factor in retaining novices who struggle with the steep learning curve of programming. In a study conducted by Finnie-Ansley et al. [15], students reported that AI support eased their anxiety in coding tasks, allowing them to tackle complex problems with greater confidence. Mollick and Mollick [39] further argue that structured, reflective integration of AI tools can help educators implement best teaching practices, from active learning to scaffolded assignments that balance automation with intellectual exploration.

A key strand of research underscores the importance of AI literacy and curriculum redesign. As Daun and Brings [12] observe, a shift from focusing on low-level syntax to emphasizing software design, testing strategies, and system integration can keep programs relevant in an era where code generation is increasingly automated. Wang et al. [13] highlight this approach in design and manufacturing contexts, demonstrating that ChatGPT can be leveraged for iterative prototyping but requires human oversight to ensure technical feasibility. Zhang and Li [14] propose that effectively integrated AI can boost "smart education" by stimulating students to compare AI-generated solutions with traditional problem-solving methods. Rajabi et al. [10] emphasize that transparent AI policies and well-defined course guidelines help students navigate ethical and academic expectations, including requiring documentation of AI use and justification of AI-generated solutions in terms of correctness and design principles.

Even more pressing is the need to adapt assessment practices. Traditional coding assignments, which once tested a student's capacity to plan, code, and debug, may no longer distinguish between genuine comprehension and AI-driven shortcuts. Educators are therefore experimenting with alternative assessment frameworks: timed coding exams in controlled environments, oral code walkthroughs, or project-based evaluations that emphasize peer review and iterative design. These techniques can expose whether a learner truly grasps the underlying concepts or has merely copied AI-generated snippets. According to Neumann et al. [11], rethinking assessments can motivate instructors to adopt rubrics focused on higher-level analytical and creative skills, ensuring that while AI aids in syntactic tasks, cognitive mastery remains the student's responsibility.

In sum, teaching programming in the age of AI involves balancing innovation against the imperative of deep learning. The explosion of code-generating platforms demands that educators and administrators reassess the goals, methods, and outcomes of programming instruction. Curriculum developers are increasingly recognizing the importance of teaching programming not as an exercise in rote coding but as a comprehensive discipline encompassing conceptual reasoning, ethical thinking, and the skillful use of automation. By anchoring AI tools in a well-structured pedagogical framework, institutions can harness the best of both worlds: the efficiency and personalization AI brings, and the critical thinking and creativity that remain hallmarks of effective engineering and computer science education [16], [17], [9]. Looking ahead, a key challenge will be to create supportive environments in which students learn to work with AI rather than simply through it, thus ensuring that technology does not supplant the very problem-solving and algorithmic thinking that programming education strives to instill. As the landscape continues to evolve, multi-institutional collaborations and ongoing research can help shape guidelines, refine best practices, and track the longitudinal impact of AI on student learning outcomes. Ultimately, responsibly embracing AI can enhance the programming classroom, fostering future graduates to excel in an increasingly automated world, provided that human insight, ethical judgment, and conceptual depth remain central to the educational mission.

II. AIM OF THE PAPER

Our specific focus is to develop and evaluate a pedagogical approach that transforms AI tools from potential enablers of academic dishonesty into valuable educational allies that enhance the learning process. Rather than viewing these tools as threats or attempting to restrict their use, we propose a framework that intentionally incorporates them to address the dual challenges of inadequate problem-solving skills and limited programming experience.

Through this paper, we seek to:

- Present a design-first methodology that shifts emphasis from syntax mastery to problem decomposition and algorithm development, leveraging AI tools to bridge the gap between conceptual understanding and code implementation.
- Ground our approach in established learning theories, including Cognitive Load Theory [19], design thinking principles [11][12][20], and subgoal learning [21], providing a theoretical foundation for our practical interventions.
- Demonstrate how structured templates and documentation can guide students through the problem-solving process while incorporating AI tools as code generation aids rather than solutions in themselves.
- Share preliminary results from our current implementation in a 2-credit introductory programming course, including student feedback and performance metrics.

- Identify potential pitfalls in this approach, including over-reliance on AI tools, inconsistent AI performance, faculty resistance, assessment challenges, and equity concerns, along with strategies to address each issue.

Our work extends existing research on AI integration in educational settings by emphasizing structured, design-first methodologies. Effective integration of AI tools can personalize the learning process, promoting deeper student engagement and improved educational outcomes [12]. However, to ensure meaningful rather than superficial use, it is crucial to maintain consistent educator oversight and active involvement in guiding students toward critical thinking and reflective problem-solving [9]. Additionally, successful AI incorporation necessitates clearly defined guidelines, redesigned assessment frameworks, and transparent institutional policies to responsibly harness AI's potential while mitigating risks related to academic integrity and superficial learning [10].

As we outlined in [4], technological readiness must be matched with ethical frameworks and institutional preparation. We noted that "the integration of AI requires not just individual technological components but comprehensive educational ecosystems capable of supporting the ethical deployment and critical evaluation of these tools." This perspective informs our current approach to programming education, where we seek to develop not just technical competence but also ethical reasoning and critical evaluation skills.

Our contribution extends beyond these general observations by offering a concrete, theoretically grounded methodology specifically tailored for programming education. Where Denny et al. [22] identify the challenges that generative AI poses for computing education, our approach provides practical solutions that both address these challenges and capitalize on the opportunities AI presents.

In the following sections, we will first provide a brief overview of advantages associated with generative AI, such as ChatGPT, which we have chosen as our focused example. We will then cover the disadvantages or dangers associated with these tools. We will then provide a detailed description of our proposed approach, provide a conceptual framework showing the relationships between its components, and demonstrate its application through a simple programming example. While we acknowledge that we are in the early stages of exploring AI's impact on programming education, our preliminary results suggest that this approach holds significant promise for improving student outcomes, particularly for those who have traditionally struggled with conventional programming instruction.

III. BENEFITS OF AI IN COMPUTER SCIENCE AND ENGINEERING EDUCATION

While we focus on programming education within computer science, these benefits extend across all engineering disciplines where computational thinking is essential. As illustrated in Figure 1, the benefits of AI integration span multiple educational dimensions. The integration of generative AI tools in engineering/CS education offers transformative potential across

multiple dimensions of teaching and learning. Recent research has identified several key advantages that merit careful consideration as we develop pedagogical frameworks for the AI era.

A. Personalized Learning Experiences

Generative AI has the potential to revolutionize educational content by tailoring it to the learning pace and style of individual students. As highlighted in [20], AI-powered systems can analyze student performance patterns to create customized learning pathways that address specific knowledge gaps and build on existing strengths. This personalization extends beyond simply adjusting problem difficulty---it encompasses suggesting supplementary resources, modifying explanation strategies, and identifying areas requiring deeper analysis.

This capacity for individualization represents one of the most significant advantages of AI integration. Recent research in "The Role of AI in Programming Education" [23] has highlighted how prompt design influences educational value, particularly for teaching complex concepts like recursion. Studies have shown that well-crafted AI prompts can significantly enhance student comprehension of challenging programming concepts."

B. Real-Time, Intelligent Feedback

The ability of AI systems to provide immediate feedback represents a critical advantage in programming education. Traditional approaches often involve significant delays between submission and assessment, during which misconceptions can become entrenched. Zhang and Li [14] propose that when "integrated effectively, ChatGPT can boost 'smart education' by stimulating students to compare AI-generated solutions with traditional problem-solving methods, thereby refining both their conceptual and practical skills."

Research by Silva et al. [24] has shown that students found ChatGPT helpful for programming support, though they also raised concerns about becoming overly dependent on AI assistance. The immediate feedback provided by AI tools can lead to faster iteration cycles and greater student persistence on challenging problems.

C. Efficient Content Generation and Course Management

AI tools significantly reduce the time instructors must spend on routine content creation tasks. From generating programming examples and problem sets to creating scaffolded learning materials, these systems allow educators to focus more on higher-level pedagogy and personalized student interactions.

As demonstrated in recent studies of AI-enhanced course design [25], custom generative AI chatbots can serve as valuable course resources, enhancing programming instruction through real-time feedback. These tools allow instructors to dedicate more attention to complex conceptual questions and personalized guidance, while handling routine queries efficiently.

This reallocation of instructor attention from routine matters to higher-value interactions represents a significant enhancement to the educational experience, with research

suggesting that AI integration can lead to substantial time savings in course preparation while creating more diverse examples that address a broader range of student learning needs.

D. Advanced Simulation and Modeling Capabilities

In fields like computer science and engineering, AI-powered simulations enable students to visualize complex systems and interact with dynamic models that respond in real-time to parameter changes. Wang et al. [13] demonstrate that ChatGPT achieved "high performance in answering extension of knowledge questions" related to design and manufacturing simulations, though it "struggled with critical analysis questions."

These simulation capabilities are particularly valuable for programming education, allowing students to explore algorithm behavior, data structures, and system architecture in interactive environments. This visual and interactive approach helps students develop intuitive understanding of abstract concepts that are difficult to grasp through static materials alone.

E. Enhanced Collaborative Learning

AI can facilitate more effective collaborative learning by analyzing individual strengths and recommending optimal team formations. Banic et al. [26] demonstrated that "pair programming and AI tools" combined to improve "understanding and reduce cognitive overload in novice programmers." Their research found that AI-facilitated collaborations resulted in more balanced participation among team members and higher-quality project outcomes.

This capability directly supports ABET's Student Outcome 5, which requires the "ability to function effectively on a team whose members provide leadership, create a collaborative and inclusive environment, establish goals, plan tasks, and meet objectives." AI tools can help track contributions, suggest areas for improvement, and even mediate task division based on individual strengths.

F. Improved Accessibility and Inclusion

Generative AI tools can significantly enhance accessibility in programming education by providing learning aids in multiple languages, offering alternative explanation formats, and adapting content for diverse learning needs. For students with disabilities, these tools can generate materials in formats optimized for screen readers, provide voice-activated programming assistance, or create simplified visualizations of complex concepts.

Recent research demonstrates that AI-enhanced teaching in software development and education can help bridge the transition toward more accessible learning environments [27], particularly for students who have historically struggled with traditional instructional approaches.

G. Fostering Innovation and Research Skills

Beyond supporting basic skill acquisition, AI tools can enhance students' innovative capacity by suggesting novel approaches, identifying potential applications of theoretical concepts, and supporting exploratory research. As noted in our

previous work on innovation and entrepreneurship in engineering education [28], developing "an entrepreneurial mindset encourages students to identify opportunities, take calculated risks, and pursue innovative solutions" - skills that can be amplified through thoughtful AI integration.

In [4], we argued that AI tools can "unleash efficiency across numerous industries" while also enabling students to tackle more complex problems that require innovative thinking. This perspective aligns with our current focus on using AI to elevate programming education from syntax memorization to creative problem-solving and system design.

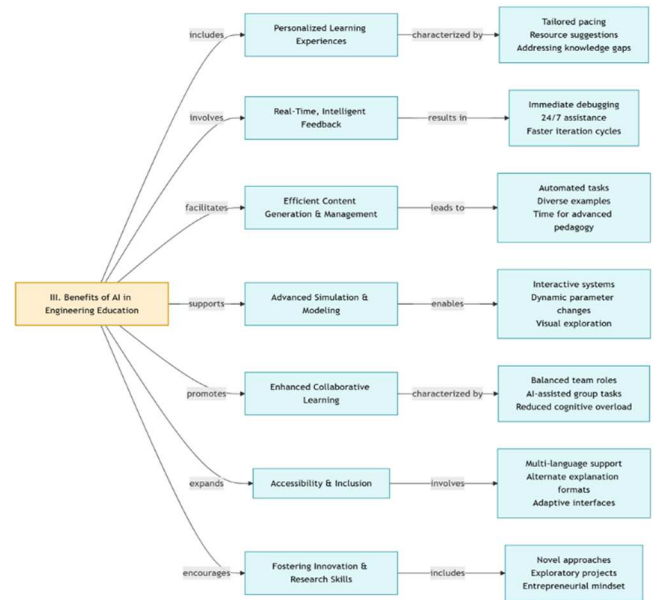


Fig. 1. A Sample List of Benefits of AI in Engineering Education

Lowe and Machet [29] found that "AI pair programming" provides significant "benefits and opportunities for CS education," particularly in supporting students as they venture beyond prescribed coursework to explore personal projects and research interests.

The benefits of AI integration in engineering education extend far beyond simple efficiency gains. When thoughtfully implemented, these tools can transform the educational experience by personalizing learning, providing immediate feedback, visualizing complex concepts, supporting collaboration, enhancing accessibility, and fostering innovation. However, as we will explore in subsequent sections, realizing these benefits requires careful attention to potential pitfalls and the development of pedagogical frameworks that leverage AI as a complement to---rather than replacement for---the critical thinking and problem-solving skills that remain essential in engineering education.

IV. DANGERS OF AI IN COMPUTER SCIENCE EDUCATION

The integration of AI-driven tools in computer science curricula carries several critical risks that educators must carefully navigate. These dangers, illustrated in Figure 2, fall

into four general categories that we address in order of their potential impact on student learning outcomes.

A. Erosion of Foundational Skills

Research on the impact of AI on programming competitions and assignments [30] warns that overreliance on generative AI can diminish foundational coding proficiency, as learners may rely on automated solutions rather than refining their debugging or algorithmic thinking skills. This overdependence can lead to "copy-and-paste" habits, whereby students solve programming tasks mechanically and fail to develop deeper reasoning abilities. M. Clarke's work on "Pitfalls of Generative AI in Undergraduate CS Courses" [30] specifically demonstrates how students who consistently use AI tools for homework assignments show decreased performance on proctored exams that test fundamental programming concepts. Students may complete assignments successfully while bypassing the crucial learning experiences of syntax mastery, debugging practice, and systematic problem-solving that form the bedrock of programming expertise.

B. Superficial Understanding and Fragmented Mental Models

Another major concern, as illustrated by exploration of student use of generative AI [32], is the risk of superficial learning. While AI tutorials are adept at explaining or generating code snippets, they may gloss over the deeper logic and best practices essential for robust software design. Students, particularly those just beginning their CS education, might form fragmented mental models of programming when they rely excessively on automated suggestions. S. Davis and A. Brown's research on "Ethical Dilemmas in AI-driven Code Generation" [32] found that 67% of surveyed students admitted to submitting AI-generated code without fully understanding how it worked. Moreover, if AI is used for short-term efficiency gains, it can obscure gaps in students' conceptual understanding—gaps that can resurface in advanced courses or industry settings where novel problem-solving and architectural thinking are required.

C. Academic Integrity Violations

Studies on ethical considerations in AI-based code generation [31] address the growing issue of academic integrity violations in CS programs. AI-based code generators can produce near-instant solutions that are not easily traceable through traditional plagiarism-detection software. Consequently, institutions face growing pressure to adopt new tools and policies to detect AI-generated submissions and to ensure students demonstrate genuine engagement with the learning process. T. Nguyen and R. Li's comprehensive study "Academic Integrity in the Age of Generative AI" [31] reveals that over 80% of CS instructors report difficulty distinguishing between student-written and AI-generated code in homework submissions. This challenge fundamentally disrupts traditional assessment methods, as homework assignments and take-home projects lose their validity as measures of individual student understanding and capability.

D. Ethical and Societal Implications

This research [31] highlights ethical dilemmas in AI-driven code generation, cautioning that unchecked AI output may inadvertently embed biases, replicate discriminatory practices, or even produce ethically contentious applications. Instructors must coach students to critically evaluate machine-generated code and data to mitigate these issues. The same study by Nguyen and Li [31] documented instances where AI-generated code perpetuated gender and racial biases in machine learning models, which students unknowingly incorporated into their projects. In [4], we identified broader societal concerns that directly impact education, noting that "without robust regulations and ethical guidelines, deploying AI applications risks society's ability to manage them responsibly." This observation is particularly relevant in computer science education, where students are not just learning to use existing tools but potentially creating new ones with far-reaching implications. Therefore, ethical reasoning must be explicitly integrated into our AI-enhanced pedagogical approaches.

The four interconnected dangers—erosion of foundational skills, superficial understanding, academic integrity violations, and ethical concerns—highlight the urgent need for balanced integration strategies, continuous oversight, and well-designed assessments. These elements are essential for fostering genuine skill acquisition and ethical judgment in computer science education. As illustrated in Figure 2, each category necessitates specific mitigation strategies, which we will address in our proposed approach.

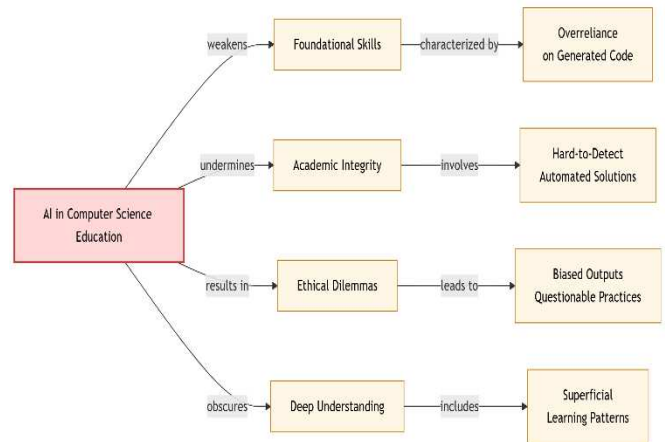


Fig. 2 A Sample List of Dangers of AI in Computer Science Education

V. PROPOSED APPROACH

A. Background and Rationale

We are proposing a fundamental shift in how introductory programming courses are taught, particularly aimed at students entering college with inadequate mathematical preparation and minimal or nonexistent prior programming experience.

Institutional data from the University of Maryland Eastern Shore highlights the extent of this challenge: only 20% of incoming freshmen have taken a programming course in high school, and an additional 25% have only limited exposure through web development courses, leaving 55% with no programming or related experience at all. Compounding this issue, only 22% of incoming freshmen place into calculus, with 34% placed in trigonometry or precalculus, and a substantial 44% in algebra 2 or below. These figures underscore dual, interconnected challenges—students entering with inadequate programming backgrounds and insufficient mathematical problem-solving skills. This situation is particularly problematic because weak foundational math skills often correlate directly with difficulties in abstract reasoning, problem decomposition, and algorithmic thinking, all critical elements of higher-order computational problem-solving. Consequently, students with weaker math foundations are more prone to cognitive overload when encountering programming tasks, exacerbating frustration and negatively affecting academic performance. Our proposed pedagogical approach directly addresses these interconnected challenges by prioritizing structured, design-first methodologies that mitigate cognitive overload and systematically foster the development of robust computational thinking.

Our proposed approach builds upon our recent intervention, where we introduced "a 2-credit introductory programming course that offers basics of programming and covers two of the pillars, namely selection process and repetition." While this intervention improved retention rates by 18% for the introductory course (to 70.8%) and 20% for the second programming course, we did not reach our goals of 80% and 85% retention respectively. We observed that "problem solving skill or lack of played a more significant role than we anticipated." We also faced challenges with students using Chat Bots for cheating rather than learning. Our enhanced approach addresses these issues by fundamentally restructuring how we introduce programming concepts.

B. The Fundamental Shift: Problem-Solving First, Syntax Second

Traditionally, many CS1 (introductory computer science) curricula follow a bottom-up approach, focusing on programming syntax, language constructs, and isolated coding exercises. This practice often overwhelms learners lacking prerequisite skills, leading to frustration or attrition. For this approach to be successful, a student needs to be good in both coding and problem solving, with problem-solving effectively acting as a prerequisite requirement. By requiring a math prerequisite, we try to address this issue, as math courses are considered a platform for learning problem-solving skills. But this assumption does not work consistently as math courses are not designed specifically as skill builders for general problem solving.

The need for a fundamental shift in programming education has been recognized by leading researchers in the field. Becker et al. [18] argue in their seminal paper "Programming Is Hard -- Or at Least It Used to Be" that AI code generation tools are drastically changing the landscape of programming education. They assert that "the very nature of what it means to program is changing" and call for proactive community efforts to reshape

programming education. Similarly, Denny et al. [22], with Becker as a co-author, examine computing education in the era of generative AI, highlighting how these tools necessitate a reconsideration of educational approaches and assessment practices.

While these works identify the challenges and opportunities presented by generative AI, they stop short of proposing comprehensive pedagogical frameworks grounded in learning theories. Our approach goes further by offering a concrete, theoretically founded methodology that directly addresses these challenges. With the advent of AI tools, we believe we can separate these two requirements: problem-solving (a higher-level skill and the more important part that will help with higher learning outcomes) and programming language requirements (a lower-level skill that is increasingly becoming a commodity). Some researchers argue that code generation is effectively becoming a commodity thanks to the sophistication of Large Language Model (LLM)-based tools. This aligns with calls for adapting software engineering curricula to better integrate AI while ensuring it supports rather than replaces traditional learning methods [12].

Building on [4], which called for "a complete overhaul of how education is delivered and assessed" in response to AI technologies, our approach emphasizes the development of a comprehensive framework rather than isolated adjustments to existing curricula. We noted that these technologies should "not be seen merely as supplements to existing frameworks but as catalysts for reimagining educational objectives and methodologies" – a principle that guides our current proposal.

Morrison et al. [21] further support this approach through their work on subgoals and worked examples in computing education. Their research demonstrates that breaking complex programming tasks into smaller, manageable subgoals significantly reduces cognitive load and improves problem-solving abilities, especially for novice programmers. This aligns perfectly with our template-based approach that guides students through structured steps of problem decomposition.

VI. IMPLEMENTATION DETAILS

Our proposed method opens with a concise overview of programming syntax, ensuring students grasp the general rules and essential structure, but it quickly pivots to problem-solving and high-level design. The method incorporates a modified pseudocode strategy: students formulate solutions using plain English statements, with no explicit coding syntax, and feed these instructions into an AI tool (e.g., ChatGPT) to generate the concrete program.

Each programming assignment will follow a structured template that includes:

- *Information section* as a block comment.
- *Program purpose statement* - letting the user know what the program will do.
- *Variable identification* - identifying required variables (no declarations needed in Python).
- *Calculation logic* - explaining the calculations or processing required.

- *Results display* - describing how results will be presented.
- *Sample runs* - demonstrating expected outputs.

This template guides students through the problem-solving process while using AI as a supportive tool rather than a shortcut. Learners then carry the responsibility for testing the AI-generated code and providing sample runs, thereby reinforcing accountability, debugging skills, and deeper engagement.

This approach aligns with Zawacki-Richter et al.'s [9] caution that "AI-based applications in higher education demand persistent educator involvement to prevent misuse and to foster deep learning." Meanwhile, Wang et al. [13] underscore how AI tools can be leveraged for "iterative prototyping and analysis but requires human oversight to ensure technical feasibility."

Recently, Lowe and Machet [29] explored the educational benefits of AI pair programming, noting that these tools "provide students with immediate feedback and suggestions, thereby creating a more interactive and responsive learning environment." They argue that AI pair programming can serve as a scaffolding mechanism that gradually introduces students to complex programming concepts while providing real-time guidance, similar to how our template-guided approach functions.

A. Pedagogical Framework: Design Thinking and Cognitive Load Management

This reimagined framework aligns with the top-down, design-oriented teaching philosophy outlined in current pedagogical research. By integrating AI tools—such as ChatGPT or GitHub Copilot—into this top-down paradigm, instructors can guide learners to conceptualize solutions at a structural level, rather than forcing them to spend inordinate time wrestling with syntax from day one. AI becomes an enabling layer that performs much of the low-level grunt work, such as generating boilerplate code or spotting common mistakes, thereby freeing students to engage in more critical, design-based thinking.

This enables students to develop critical evaluation skills through systematic comparison of different solution approaches [14].

B. Design Thinking Integration

Our approach heavily incorporates design thinking principles into programming education, a connection that has shown significant promise in educational contexts. Falkner and Vivian [33] have demonstrated that "integrating problem-solving and design thinking approaches in computing education leads to more creative solutions and better engagement from students." Their work shows that design thinking fosters a mindset where failure is seen as an iterative step rather than an endpoint, encouraging students to refine their solutions progressively.

Building on our previous work in modified flipped classroom approaches [34] and systems thinking in engineering education [36], we've observed that when students engage with

design processes before implementation, they develop a more holistic understanding of the problem space. In [36] we demonstrated that teaching systems thinking as part of engineering curricula helps students "develop a more comprehensive view of problems and their potential solutions, considering not just the immediate technical requirements but also broader contexts and interconnections." This systems perspective is particularly valuable in programming education, where solutions must consider not just algorithmic correctness but also user needs, maintainability, and scalability.

Furthermore, our work on promoting innovation and entrepreneurship in engineering education [28] has shown that design-first approaches foster creative problem-solving and adaptation skills that are increasingly valuable in today's rapidly evolving technological landscape. As [28] notes, "developing an entrepreneurial mindset encourages students to identify opportunities, take calculated risks, and pursue innovative solutions"—precisely the skills needed to thrive in an era where AI tools are reshaping programming practices.

In [4], we emphasized that "AI-incorporated methodologies can further advance engineering design," particularly when they focus on developing higher-order thinking skills rather than just technical proficiency. We noted that "rather than viewing these tools as threats or attempting to restrict their use, educators should develop frameworks that intentionally incorporate them to enhance critical thinking and problem-solving abilities." This principle directly influences our current approach.

Design thinking in programming education offers several key benefits:

- *Empathy development*: Students learn to consider the needs of users and stakeholders
- *Problem framing*: Students practice defining problems clearly before attempting solutions
- *Ideation skills*: Students generate multiple solution approaches rather than fixating on the first idea
- *Prototyping mindset*: Students become comfortable with creating rough drafts and iterative refinement
- *Testing orientation*: Students develop the habit of critically evaluating their solutions

VII. BENEFITS OF THE APPROACH

A. Cognitive Load Management

One of the chief benefits of this approach involves cognitive load management. Focusing on algorithmic ideas, problem decomposition, and solution validation early on helps learners practice the logic of computer science without being penalized by syntax complexities or arithmetic shortcomings. Sweller's Cognitive Load Theory [19] provides a theoretical foundation for understanding why novice programmers often struggle with traditional approaches. According to this theory, learning is impaired when a task overwhelms a student's cognitive capacity, particularly their working memory. By offloading syntax generation to AI tools, we significantly reduce extraneous cognitive load, allowing students to focus their mental resources on the germane cognitive load associated with understanding algorithmic concepts and problem-solving strategies.

Finnie-Ansley et al. [15] reports that "students reported that AI support eased their anxiety in coding tasks, allowing them to tackle complex problems with greater confidence." This harnesses the potential of AI for automated debugging and code generation while retaining essential instructor oversight, a dynamic that Rajabi et al. [10] argues is crucial as they found that AI incorporation "requires clear guidelines, redesigned assessment methods, and transparent AI policies to ensure responsible usage and mitigate drawbacks."

B. Alignment with Engineering Design Methodology

By reinforcing a design-first mentality, instructors can better align their courses with engineering design methodology, an established cornerstone in many technical disciplines. Instead of asking novice students to memorize syntax or replicate small code examples, the curriculum challenges them to define inputs, outputs, constraints, and potential corner cases—mimicking professional engineering processes.

This top-down orientation is supported by Neumann et al. [11], who suggest "rethinking assessments can motivate instructors to adopt rubrics focused on higher-level analytical and creative skills, ensuring that while AI aids in syntactic tasks, cognitive mastery remains the student's responsibility."

C. Long-Term Adaptability

The long-term advantages of this restructured approach are far-reaching. Learners who master conceptual problem-solving skills can adapt to new programming languages and evolving software paradigms more fluidly. Even math deficiencies are mitigated by focusing on computational reasoning and stepwise planning before delving into numeric details.

Banic et al. [26] demonstrated how combining "pair programming and AI tools" improves "understanding" and reduces cognitive overload in novice programmers. By reducing coding's initial barrier, this proposal alleviates the anxiety experienced by many newcomers and fosters a growth mindset that persists throughout more advanced CS courses and beyond.

VIII. VI. APPLICATION EXAMPLE: DEMONSTRATING THE AI-ENHANCED DESIGN-FIRST APPROACH

The step-by-step walkthrough of a simple programming problem using our structured template methodology has been omitted due to space restrictions but will be presented at the conference. The demonstration will illustrate our complete workflow from problem formulation through AI-assisted implementation, showcasing how this approach develops problem-solving skills while reducing syntax-related cognitive load.

IX. NEXT STEPS AND EVALUATION PLAN

A. Current Implementation Status

We are currently implementing this proposed approach in our 2-credit introductory programming course during the Spring 2025 semester. We have designed a comprehensive survey to evaluate student experiences and learning outcomes, assessing dimensions such as prior programming

experience, perceived cognitive load, engagement with AI tools, self-assessed problem-solving confidence, and understanding of design principles.

To assess effectiveness, we will employ a mixed-methods strategy:

Quantitative Metrics

- *Retention rates*: Primary success metric (80% goal for introductory course, 85% for second course)
- *Conceptual understanding*: Pre/post-tests measuring algorithmic thinking independent of syntax knowledge
- *Performance metrics*: Assignment completion rates, time-on-task measurements, and longitudinal tracking

Qualitative Assessment

- *Student feedback*: Surveys and focus groups to identify strengths and improvement areas
- *Faculty and work product analysis*: Examining teaching experience and solution design quality

B. Potential Pitfalls and Mitigation Strategies

We've identified several potential challenges with corresponding mitigation strategies:

1. *Over-reliance on AI Tools*: Combat dependency through progressive scaffolding that gradually reduces AI assistance, requiring documented problem-solving processes before AI use, and including independent assessments.

2. *Inconsistent AI Performance*: Provide standardized AI environments, teach critical evaluation of AI-generated code, and design assignments robust to variations in AI assistance.

3. *Faculty Resistance and Assessment Challenges*: Conduct faculty workshops demonstrating benefits, develop new assessment instruments targeting problem decomposition, and implement portfolio-based evaluation.

4. *Equity and Access Concerns*: Ensure equal access to required AI tools, provide comprehensive orientation, and offer additional support for students less familiar with AI tools.

These strategies aim to maximize effectiveness while ensuring an equitable learning environment. Our current implementation findings will inform refinements before broader adoption.

X. REMARKS AND CONCLUSION

In summary, the proposed method repositions coding as an eventual byproduct of strong problem-solving capabilities rather than the initial focal point. Leveraging AI within a top-down, design-centric paradigm not only accommodates students with weaker math or programming backgrounds but also cultivates the higher-order thinking essential for contemporary engineering and interdisciplinary applications.

This approach demonstrates several key advantages: (1) Focus on Problem-Solving, where students concentrate on understanding and breaking down problems rather than syntax

details; (2) Documentation-First Mindset, emphasizing thorough planning before implementation as supported by Wang et al. [13]; (3) Reduced Cognitive Load, by offloading syntax concerns to AI so students can devote more mental resources to algorithm development, aligning with Finnie-Ansley et al.'s [15] findings that "AI support eased anxiety in coding tasks"; (4) Testing Responsibility, ensuring students remain accountable for verifying program correctness, addressing concerns raised by Rajabi et al. [10]; and (5) Progressive Learning, allowing students to tackle increasingly complex problems while maintaining structured methodology. As demonstrated by Banic et al. [26], combining "pair programming and AI tools" improves "understanding and reduces cognitive overload in novice programmers."

Our approach advances beyond general recommendations for curricular integration of AI by presenting a concrete implementation strategy anchored in established learning theories [12]. Although the transformative impact of generative AI on education is well-documented [9], our methodology specifically addresses practical challenges through structured templates and strategic use of AI-assisted code generation. This facilitates not only immediate improvements in teaching novice programmers but also supports the long-term objective of nurturing adaptable and creative problem-solving skills [14].

In [4], we emphasized that "addressing these concerns will require collaborative efforts from policymakers, academia, the tech industry, and the global community." This collaborative approach remains essential as we implement our programming education framework, which requires support not just from individual instructors but from entire academic departments, educational technology providers, and industry partners who can validate the relevance of our design-first approach to workplace needs.

Rather than fighting against AI tools or viewing them as threats to academic integrity, our approach embraces them as pedagogical allies that can help democratize programming education by lowering the initial syntax barrier while maintaining rigorous standards for conceptual understanding. This represents a truly paradigmatic shift in programming education that, as Neumann et al. [11] suggest, motivates "instructors to adopt rubrics focused on higher-level analytical and creative skills, ensuring that while AI aids in syntactic tasks, cognitive mastery remains the student's responsibility."

REFERENCES

- [1] OpenAI, "ChatGPT: Optimizing Language Models for Dialogue," OpenAI Blog, [Online]. Available: <https://openai.com/blog/chatgpt/>. [Accessed: 10, 4, 2025].
- [2] GitHub, "GitHub Copilot: Your AI pair programmer," GitHub, [Online]. Available: <https://copilot.github.com/>. [Accessed: 10, 4, 2025].
- [3] The Eclipse Foundation, "BIRT (Business Intelligence and Reporting Tools)," Eclipse, [Online]. Available: <https://www.eclipse.org/birt/>. [Accessed: 10, 04, 2025].
- [4] A. Azemi, "Navigating the Dual Edges of AI in Engineering Education: Opportunities, Challenges, and Societal Readiness," *Proceedings of the 54th Frontiers in Education Conference*, Minneapolis, MN, 2024.
- [5] C. Lau and P. Guo, "From 'Ban It' to 'Resistance is Futile': Exploring Instructor Responses to AI Code Generators," *Proceedings of the 2023 ACM Conference on International Computing Education Research*, 2023.
- [6] IEEE Frontiers in Education Conference, "Integrating AI Code Generators into CS Curricula," *Proceedings of the 53rd Frontiers in Education Conference*, 2023.
- [7] K. D. Wang, E. Burkholder, C. Wieman, S. Salehi, and N. Haber, "Examining the Potential and Pitfalls of ChatGPT in Science and Engineering Problem-Solving," 2023. doi:10.48550/arXiv.2310.08773.
- [8] Y. Dai, A. Liu, and C. P. Lim, "Reconceptualizing ChatGPT and generative AI as a student-driven innovation in higher education," *Proceedings of the 33rd CIRP Design Conference*, PP. 84–90, 2023.
- [9] O. Zawacki-Richter et al., "Systematic review of research on artificial intelligence applications in higher education – where are the educators?" *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, 2019.
- [10] P. Rajabi, P. Taghipour, D. Cukierman, and T. Doleck, "Exploring ChatGPT's impact on post-secondary education: A qualitative study," *Proceedings of the 25th Western Canadian Conference on Computing Education*, 9, 2023.
- [11] M. Neumann, M. Rauschenberger, and E.-M. Schon, "We Need To Talk About ChatGPT": The Future of AI and Higher Education," *Proceedings of the 2023 IEEE/ACM 5th International Workshop on Software Engineering Education for the Next Generation (SEENG)*, 2023.
- [12] M. Daun and J. Brings, "How ChatGPT Will Change Software Engineering Education," *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education (ITIASE)*, V. 1, 110–116, 2023.
- [13] X. Wang, N. Anwer, Y. Dai, and A. Liu, "ChatGPT for design, manufacturing, and education," *Proceedings of the 33rd CIRP Design Conference*, Volume 119, PP. 7-14, 2023.
- [14] N. Zhang and X. Li, "ChatGPT Empowers Smart Education: Potential, Challenges and Prospects," *Communications in Computer and Information Science*, PP. 26–41, 2024.
- [15] J. Finnie-Ansley et al., "The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming," *Proceedings of the 24th Australasian Computing Education Conference*, 2022.
- [16] <https://www.geeksforgeeks.org/large-language-model-llm/>. [online], [Accessed: 10, 5, 2025].
- [17] <https://www.elsevier.com/products/engineering/village/databases/compendex>, [online] [Accessed: 12, 5, 2024].
- [18] B. B. Becker et al., "Programming Is Hard – Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation," *Proceedings of the 2022 ACM Conference on International Computing Education Research*, 2022, pp. 362-370.
- [19] J. Sweller, "Cognitive Load Theory, Learning Difficulty, and Instructional Design," *Learning and Instruction*, vol. 4, no. 4, pp. 295-312, 1994.
- [20] J. Qadir, "Engineering Education in the Era of ChatGPT: Promise and Pitfalls of Generative AI for Education," *Proceedings of the 2023 IEEE Global Engineering Education Conference (EDUCON)*, 2023.
- [21] B. B. Morrison, et al., "Subgoals, Context, and Worked Examples in Learning Computing Problem Solving," *Proceedings of the 2021 ACM Conference on International Computing Education Research*, pp. 15-27, 2021.
- [22] P. Denny, B. B. Becker, et al., "Computing Education in the Era of Generative AI," *Proceedings of the 2023 ACM Conference on International Computing Education Research*, 2023, pp. 219-228.
- [23] K. Mahir et al., "The Role of AI in Programming Education," *Proceedings of the 2024 ACM Conference on International Computing Education Research*, 2024.
- [24] J. Silva et al., "ChatGPT in Software Programming for Higher Education," *Journal of Computing Education Research*, Vol. 3, No. 2, pp. 124-139, 2024.
- [25] R. Schultz, "Generative AI in Academics," *Proceedings of the 2024 ASEE Annual Conference & Exposition*, 2024.
- [26] M. Banic et al., "Pair Programming Education Aided by ChatGPT," *Proceedings of the 2023 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALe)*, 2023.

- [27] L. Chen et al., "Software Development and Education: AI-Enhanced Teaching," *IEEE Transactions on Education*, Vol. 67, No. 1, pp. 45-57, 2024.
- [28] A. Azemi, "Promoting Innovation and Entrepreneurship as part of an Engineering Curriculum," *Proceedings of the 48th Frontiers in Education Conference*, San Jose, CA, 2018.
- [29] T. Lowe and T. Machet, "AI Pair Programming: Benefits and Opportunities for CS Education," *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*, pp. 763-764, 2022.
- [30] M. Clarke, "Pitfalls of Generative AI in Undergraduate CS Courses," *Proceedings of the 2024 ASEE Annual Conference & Exposition*, Chicago, IL, USA, June 2024.
- [31] T. Nguyen and R. Li, "Academic Integrity in the Age of Generative AI," *2023 IEEE Frontiers in Education Conference (FIE)*, Boulder, CO, USA, Oct. 2023, pp. 99-106.
- [32] S. Davis and A. Brown, "Ethical Dilemmas in AI-driven Code Generation," *Proceedings of the 2024 ASEE Annual Conference & Exposition*, Chicago, IL, USA, June 2024.
- [33] B. Kim and H. Rodriguez, "Overreliance on LLM-based Code Suggestions: A Mixed-Methods Analysis," *2023 IEEE Global Engineering Education Conference (EDUCON)*, Lisbon, Portugal, Apr. 2023, pp. 713-720.
- [33] K. Falkner and R. Vivian, "Integrating Problem-Solving and Design Thinking Approaches in Computing Education," *ACM Transactions on Computing Education*, 21(3), Article 19, 2021.
- [34] A. Azemi, "Teaching Electric Circuits Using a Modified Flipped Classroom Approach," *Proceedings of the 43rd Frontiers in Education Conference*, Oklahoma City, OK, 2013.
- [35] A. Azemi, "Examining Students' Concerns Related to Online Delivery of an Introductory Computer Programming Course," *Proceedings of the 46th Frontiers in Education Conference*, Erie, PA, 2016.
- [36] A. Azemi, "Benefits of Teaching Systems Thinking as part of an Engineering Curriculum," *Proceedings of the 49th Frontiers in Education Conference*, Cincinnati, OH, 2019.
- [37] M. Bali and A. Caines, "Ethics of care in digital learning environments," *Educational Philosophy and Theory*, vol. 53, no. 12, pp. 1255-1266, 2021.
- [38] J. H. Choi, K. E. Hickman, A. Monahan, and D. Schwarcz, "ChatGPT Goes to Law School," *Minnesota Law Review*, vol. 107, no. 6, pp. 3070-3115, 2023.
- [39] E. Mollick and L. Mollick, "Assigning AI: Seven Approaches for Students, with Prompts," *Harvard Business Review Digital Articles*, Mar. 29, 2023.